

Developer Coding Challenge - PHP

Commercial in confidence

Overview

Build a small **User Import Application**

The application should allow users to import a CSV file through both a **web UI** and a **CLI**.

The import flow is:

HTML

```
Upload -> Parse -> Validate -> Preview -> Import
```

The same import logic should be shared between the UI and CLI where practical.

User Data

The CSV file contains three columns:

HTML

```
name,surname,email  
John,Smith,john@example.com  
Jane,Doe,jane@example.com
```

The CSV may contain an arbitrary number of users.

The application should:

- Parse the CSV file.
- Capitalise the name and surname fields before importing.
- Convert email addresses to lowercase.
- Validate email addresses before importing.
- Reject invalid records.
- Ensure email addresses are unique.
- Report validation errors clearly.
- Handle errors gracefully.

For example:

HTML

```
john,smith,JOHN@EXAMPLE.COM
```

Should become:

```
HTML
John,Smith,john@example.com
```

An invalid email such as:

```
HTML
john,smith,john@example.com@example.com
```

should not be imported.

Web UI

Build a simple React interface that allows the user to:

1. Upload a CSV file.
2. Parse and validate the file.
3. Preview the results.
4. Import the valid records.
5. See the final import result.

The preview should make it clear which records are valid and which records contain errors.

For example:

```
HTML
Users found: 100
Valid:      95
Invalid:    5

| Name | Surname | Email           | Status |
| John | Smith   | john@example.com | Valid  |
| Jane | Doe     | invalid-email    | Error  |

[ Import 95 users ]
```

The UI does not need to be visually complex. We are primarily interested in a clear, usable and maintainable implementation.



CLI

The application must also provide a command-line interface for importing CSV files.

For example:

```
Shell
php user_upload.php --file users.csv
```

The CLI should support the following options:

```
HTML
--file <filename>    CSV file to process
--dry-run            Parse and validate without importing
--create-table        Create/rebuild the users table
--help               Display available options
```

--dry-run should perform all parsing and validation but must not modify the database.

For example:

```
Shell
php user_upload.php --file users.csv --dry-run
```

should report the records that would be imported without inserting them into the database.

Database

The application should use PostgreSQL.

The users table should contain at least:

```
HTML
name
surname
email
```

The **email** field must be unique.

The application should provide a way to create/rebuild the table as described by the

--create-table CLI option.

Database connection details should be configurable rather than hard-coded.



Error Handling

The application should gracefully handle common errors, including:

- Missing or unreadable CSV files.
- Invalid CSV data.
- Missing required fields.
- Invalid email addresses.
- Duplicate email addresses.
- Database errors.
- Invalid command-line arguments.

Validation errors should identify the affected record where practical.

Source Control

All of your work on this coding challenge must be managed using git as the version control system, with the repository made available via an online repository: GitHub (github.com), Bitbucket (bitbucket.org), etc. This is how you will deliver your final code once you have completed the challenge.

The repository should demonstrate your development process rather than containing only a single final commit.

Documentation

Please include a README containing:

- Application overview.
- Requirements.
- Installation/setup instructions.
- Database configuration.
- How to start the application.
- How to use the web UI.
- How to use the CLI.
- Examples of CLI commands.
- Any assumptions or design decisions you made.

Technology



The expected technology stack is:

- PHP 8.3+
- PostgreSQL
- React
- Git

You may use appropriate libraries or frameworks. Any dependencies should be documented in the README.

Expected Effort

You do not need to implement every possible improvement. Prioritise:

- Correctness
- Simplicity
- Maintainability
- Clear documentation

What We Are Looking For

The challenge is intended to assess your ability to:

- Build a working PHP application.
- Build a simple React UI.
- Work with PostgreSQL.
- Design and implement an import/validation flow.
- Handle errors appropriately.
- Write maintainable code.
- Write useful tests.
- Use Git effectively.
- Explain your implementation decisions.

Questions

The aim of this task is to test both your development skills as well as simulate a real world project task. Guidance can be sought regarding the requirements and deliverables of this task, however questions on "how to do it" won't be accepted.

